

Security Audit

Report for Starcore - relay - billing

Date: July 3, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About the Audit Target	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Security Issues	2
1.3.2 Additional Recommendation	3
1.4 Security Model	3
Chapter 2 Findings	4
2.1 Recommendation	4
2.1.1 Add a zero-address check	4
2.1.2 Align comments with implementation	5
2.1.3 Fix the length of storage gaps	5
2.1.4 Align burn access control with documentation	6
2.2 Note	6
2.2.1 Potential centralization risks	6
2.2.2 Proxy deployment and implementation binding should be atomic	7
2.2.3 Off-chain signing and accounting dependency	7
2.2.4 EIP-1153 deployment compatibility	7

Report Manifest

Item	Description
Client	Norewsin - web
Target	Starcore - relay - billing

Version History

Version	Date	Description
1.0	July 3, 2026	First release

Signature

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository ¹ of Starcore-relay-billing of Norewsin-web.

Starcore Relay Billing is a fixed-supply token billing and settlement system for Starcore GPU compute and model-relay services. The protocol uses StarcoreToken (SC), a burnable ERC-20 with a one-time 1 billion token mint to the treasury as the unit of the account. Billing-Vault instances custody SC deposits and settle payouts: users top up by depositing SC, while withdrawals are authorized off-chain by a backend signer and verified on-chain through EIP-712 signatures. The vault does not track per-user balances on-chain. Instead, an off-chain backend acts as the authoritative ledger and uses deposit and withdrawal events for reconciliation. Administrative controls include signer rotation, pause controls, withdrawal limit configuration, signature validity configuration, emergency fund withdrawal, and UUPS upgrades controlled by the owner multisig.

Note this audit only focuses on the smart contracts in the following directories/files:

- src

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (Version 1), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (Version 0), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
starcore-relay-billing	Version 1	507c99db6854dd56ff68c1f94cfc12fc3d246f03
	Version 2	ead37789976503d0e1fbd07bf51910759ad9d4f5

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on,

¹<https://github.com/norewsin-web/starcore-relay-billing>

the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

- * Access control
- * Permission management
- * Whitelist and blacklist mechanisms
- * Initialization consistency
- * Improper use of the proxy system
- * Reentrancy
- * Denial of Service (DoS)
- * Untrusted external call and control flow
- * Exception handling
- * Data handling and flow
- * Events operation
- * Error-prone randomness
- * Oracle security
- * Business logic correctness
- * Semantic and functional consistency

- * Emergency mechanism
- * Economic and incentive impact

1.3.2 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology² and Common Weakness Enumeration³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we have **four** recommendations and **four** notes.

- Recommendation: 4
- Note: 4

ID	Severity	Description	Category	Status
1	-	Add a zero-address check	Recommendation	Fixed
2	-	Align comments with implementation	Recommendation	Fixed
3	-	Fix the length of storage gaps	Recommendation	Fixed
4	-	Align burn access control with documentation	Recommendation	Fixed
5	-	Potential centralization risks	Note	-
6	-	Proxy deployment and implementation binding should be atomic	Note	-
7	-	Off-chain signing and accounting dependency	Note	-
8	-	EIP-1153 deployment compatibility	Note	-

The details are provided in the following sections.

2.1 Recommendation

2.1.1 Add a zero-address check

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract [BillingVault](#), the function [withdraw\(\)](#) accepts [recipient](#) as the destination address but does not validate that it is not the zero address. Since the underlying token is a standard ERC-20, a transfer to [address\(0\)](#) will revert at the token transfer step. It is suggested that the vault enforce this input constraint itself rather than rely on a token-level revert.

```
153 function withdraw(address recipient, uint256 amount, uint256 nonce, uint256 deadline, bytes
    calldata signature)
154     external
155     whenNotPaused
156     nonReentrant
157 {
158     require(amount > 0, "BillingVault: zero amount");
159     require(block.timestamp <= deadline, "BillingVault: expired");
160     require(deadline <= block.timestamp + maxSignatureValidity, "BillingVault: deadline too far
    ");
161     require(!usedNonce[nonce], "BillingVault: nonce used");
162
163     bytes32 structHash = keccak256(abi.encode(WITHDRAW_TYPEHASH, recipient, amount, nonce,
    deadline));
164     address recovered = ECDSA.recover(_hashTypedDataV4(structHash), signature);
```

```
165     require(recovered == signer, "BillingVault: invalid signature");
166     require(amount <= maxSingleWithdraw, "BillingVault: over single limit");
167
168     uint256 day = currentDay();
169     require(dailyWithdrawn[day] + amount <= maxDailyWithdraw, "BillingVault: over daily limit")
170         ;
171     usedNonce[nonce] = true;
172     dailyWithdrawn[day] += amount;
173     totalWithdrawn += amount;
174     token.safeTransfer(recipient, amount);
175     emit Withdrawn(recipient, amount, nonce);
176 }
```

Listing 2.1: src/BillingVault.sol

Suggestion Add a zero-address check for `recipient` in function `withdraw()`.

2.1.2 Align comments with implementation

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The contract `BillingVault` states that it exposes exactly two cash-flow paths (line 17), the functions `deposit()` and `withdraw()`. However, the contract also implements the owner-only function `emergencyWithdraw()`, which can transfer the vault balance during incident response. It is recommended to align the comment with the implementation.

```
15/// @notice Minimal UT custody + settlement vault. One instance is deployed per
16///     billing system (GPU compute and model relay use separate proxies with
17///     identical code). It exposes exactly two cash-flow paths: 'deposit'
18///     (top-up) and 'withdraw' (signature-verified payout).
```

Listing 2.2: src/BillingVault.sol

```
207 /// @notice Sweep the entire UT balance to 'to' (incident response only).
208 function emergencyWithdraw(address to) external onlyOwner {
209     require(to != address(0), "BillingVault: zero recipient");
210     uint256 balance = token.balanceOf(address(this));
211     token.safeTransfer(to, balance);
212     emit EmergencyWithdraw(to, balance);
213 }
```

Listing 2.3: src/BillingVault.sol

Suggestion Update the comment to describe the functions `deposit()` and `withdraw()` as normal user cash-flow paths.

2.1.3 Fix the length of storage gaps

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In upgradeable contracts, the array `__gap` usually has a length of `(50 - occupied slot)` to reserve space for future upgrades. The contract `BillingVault` has 10 slots occupied by the storage variables. However, the array `__gap` is declared with a length of 48, which does not align with the standard upgradeable storage pattern.

```
97  /// @dev Reserved storage slots for future upgrades (append new vars above this gap).
98  // forge-lint: disable-next-line(mixed-case-variable)
99  uint256[48] private __gap;
```

Listing 2.4: `src/BillingVault.sol`

Suggestion Adjust the `__gap` length to conform to the standard 50-slot layout.

2.1.4 Align burn access control with documentation

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description Contract `StarcoreToken` inherits from `ERC20Burnable`, which exposes two public functions, `burn()` and `burnFrom()`. With these functions, any holder can burn tokens from their own balance, and any approved spender can burn tokens from another holder's balance. However, the annotation states that burning is intended to be performed only by the project-party burn wallet (line 11). It is suggested that the project align the implementation with the documented intent.

```
11/// @dev Burnable so the project-party burn wallet can destroy tokens on-chain
12/// after pulling them out of a BillingVault.
13contract StarcoreToken is ERC20, ERC20Burnable {
```

Listing 2.5: `src/StarcoreToken.sol`

Suggestion Adjust the burn access control to match the documented intent.

2.2 Note

2.2.1 Potential centralization risks

Introduced by [Version 1](#)

Description In this project, several privileged roles (e.g., owner) can conduct sensitive operations, which introduces potential centralization risks. For example, the owner of the contract `BillingVault` can pause deposits and withdrawals, rotate the backend signer, update withdrawal limits, update the maximum signature validity window, execute emergency sweeps, and authorize UUPS upgrades. The backend signer can authorize withdrawals within the configured on-chain limits. If the private keys or operational controls for the privileged accounts are lost or maliciously exploited, it could affect protocol availability, withdrawal settlement, emergency custody handling, or upgrade behavior.

Feedback from the project The project confirmed that the ownership is held by a multisig, with withdrawals constrained by on-chain limits and the pause functionality.

2.2.2 Proxy deployment and implementation binding should be atomic

Introduced by [Version 1](#)

Description The deployment script deploys each `BillingVault` proxy through `Upgrades.deployUUPSProxy()` with encoded initializer calldata for the function `BillingVault.initialize()`. This pattern binds proxy deployment and initialization within the deployment flow. Future deployment scripts should preserve this approach and avoid deploying an uninitialized proxy that is initialized in a later externally submitted transaction.

Feedback from the project The project confirmed that the deploy script uses the function `deployUUPSProxy()`, which deploys and initializes atomically via initializer calldata. Future scripts will preserve this approach.

2.2.3 Off-chain signing and accounting dependency

Introduced by [Version 1](#)

Description The contract `BillingVault` does not maintain per-user balances. As a result, user balance accounting, withdrawal eligibility, and deposit reconciliation all rely on the off-chain backend as the source of truth, and the project should ensure its correctness.

This creates two operational dependencies. First, the backend must generate globally unique withdrawal nonces for each vault. If two valid withdrawals are signed with the same nonce, the first executed withdrawal consumes the nonce, and the later one reverts, even if it is for a different recipient. Second, the backend must correctly reconcile deposits and withdrawals because the contract only tracks aggregate values by state variables `totalDeposited` and `totalWithdrawn`.

Feedback from the project The project confirmed that nonce uniqueness per vault and reconciliation via `totalDeposited` or `totalWithdrawn` are documented in the backend integration guide.

2.2.4 EIP-1153 deployment compatibility

Introduced by [Version 1](#)

Description The contract `BillingVault` inherits from `ReentrancyGuardTransient`. The functions `deposit()` and `withdraw()` apply the modifier `nonReentrant`, which relies on transient storage. To ensure functionality, the vault deployment scope should remain limited to chains compatible with EIP-1153.

Feedback from the project The project confirmed that deployments are limited to [Cancun / EIP-1153-enabled](#) chains, with Polygon PoS as the production target and Sepolia for testing.

